AI Mode All Images Videos News More

in python 3.6, use ProcessPoolExecutor to make a class that can accept tasks dynamically and uses a queue to maintain ...

In Python 3.6, the `concurrent.futures.ProcessPoolExecutor` provides a high-level interface for asynchronously executing tasks using a pool of worker processes. It internally manages a queue of tasks, automatically reusing worker processes to execute those tasks as they become available. Stack Overflow +3

To meet your requirements, you can create a class that wraps the executor and uses the `subprocess` module to execute command-line strings. Python documentation +2

python

```python
import subprocess
from concurrent.futures import ProcessPoolExecutor

class CommandRunner:
    def __init__(self, max_workers=None):
        # The executor maintains its own internal task queue and worke
        # It reuses processes to execute submitted tasks.
        self.executor = ProcessPoolExecutor(max_workers=max_workers)
        self.futures = []

    def _execute_command(self, cmd_string):
        """Helper method to run the command string in a separate proce
        try:
            # Use subprocess to run the command line string.
            # shell=True is needed to execute string-based shell comma
            result = subprocess.run(
                cmd_string,
```

Ask anything

```
        """Dynamically accept and queue a new command task."""
        # submit() schedules the task and returns a Future object imme
        future = self.executor.submit(self._execute_command, cmd_stri
        self.futures.append(future)
        return future

    def shutdown(self, wait=True):
        """Clean up the executor resources."""
        self.executor.shutdown(wait=wait)

# Usage Example
if __name__ == "__main__":
    runner = CommandRunner(max_workers=4)

    # Dynamically adding tasks
    runner.submit_task("echo 'Task 1 running'")
    runner.submit_task("ls -l")
    runner.submit_task("sleep 2 && echo 'Task 3 finished'")

    # Shutdown the runner and wait for all tasks in the queue to fini
    runner.shutdown(wait=True)

    for f in runner.futures:
        print(f.result())
```

Use code with caution.

## Key Components

- **Dynamic Task Acceptance**: The `submit_task` method allows you to add commands at any time.
- **Internal Queueing**: `ProcessPoolExecutor` handles the queueing of tasks that exceed the `max_workers` count.

AI Mode All Images Videos News More

9 sites

multiprocessing — Process-based parallelism — Python 3.14 ...

17 Feb 2026 — concurrent. futures. ProcessPoolExecutor offers a higher level interface to push tasks to a background process witho...

Python documentation

ProcessPoolExecutor Class in Python - GeeksforGeeks

23 Jul 2025 — Prerequisite - Multiprocessing. It allows parallelism of code and the Python language has two ways to achieve its 1st is via multi...

GeeksforGeeks

Python ProcessPoolExecutor: 7-Day Crash Course - Medium

10 Dec 2023 — Lesson 05: Wait For All Async Tasks In The ProcessPoolExecutor. We can issue many asynchronous tasks to th...

Medium

Show all

add a notification mechanism so client can be notified when a task is finished, and the output of the command is returned.

To add a notification mechanism, the most efficient way in Python 3.6 is to use the `add_done_callback` method on the `Future` object returned by the executor. This allows you to attach a function that triggers automatically as soon as the task completes.

Here is the updated class with a callback system:

```
        """Worker function executed in the subprocess."""
        result = subprocess.run(
            cmd_string,
            shell=True,
            stdout=subprocess.PIPE,
            stderr=subprocess.PIPE,
            universal_newlines=True
        )
        return {"cmd": cmd_string, "stdout": result.stdout, "stderr":

    def submit_task(self, cmd_string, notification_callback=None):
        """
        Submits a command and optionally attaches a callback.
        The callback receives the future object as its only argument.
        """
        future = self.executor.submit(self._execute_command, cmd_strin

        if notification_callback:
            # The callback is triggered automatically when the task f
            future.add_done_callback(notification_callback)

        return future

    def shutdown(self):
        self.executor.shutdown(wait=True)

# --- Client Implementation ---

def my_notifier(future):
    """This function runs when a task is done."""
    result = future.result()
    print(f"\n[NOTIFY] Task Finished: {result['cmd']}")
    print(f"[OUTPUT] {result['stdout'].strip()}")

if __name__ == "__main__":
    runner = CommandRunner(max_workers=2)
```

## Key Enhancements:

- `add_done_callback`: This eliminates the need for the client to poll or wait manually. The executor's internal thread handles calling this function once the process returns a result.
- **Structured Return**: The `_execute_command` now returns a dictionary. This ensures the **notification callback** has access to the original command, the output, and any errors.
- **Decoupling**: The `CommandRunner` doesn't need to know what the notification does; the client just passes any callable (function or method).

Would you like to explore how to handle **thread safety** if your callback needs to update a shared **UI or database**?

AI responses may include mistakes. Learn more

make a test that submits tasks periodically for potentially long-running commands for example to sleep for 5 seconds.

To test the `CommandRunner` with periodic, long-running tasks, you can use a simple loop with `time.sleep()` in the main thread to submit tasks at specific intervals.

In the following example, the main thread acts as a "producer" that submits a new 5-second sleep command every 2 seconds. Because the `max_workers` is set to 3, the tasks will overlap and queue up, demonstrating the pool's ability to manage concurrent long-running processes.

```
    def _execute_command(self, cmd_string):
        # Executes the command-line string in a shell
        result = subprocess.run(
            cmd_string,
            shell=True,
            stdout=subprocess.PIPE,
            stderr=subprocess.PIPE,
            universal_newlines=True  # Required for string output in 
        )
        return {"cmd": cmd_string, "stdout": result.stdout, "stderr":

    def submit_task(self, cmd_string, callback=None):
        future = self.executor.submit(self._execute_command, cmd_stri
        if callback:
            future.add_done_callback(callback)
        return future

    def shutdown(self):
        self.executor.shutdown(wait=True)

# --- Test Notification Callback ---
def task_finished_callback(future):
    res = future.result()
    timestamp = time.strftime("%H:%M:%S")
    print(f"[{timestamp}] FINISHED: '{res['cmd']}'")

# --- Periodic Submission Test ---
if __name__ == "__main__":
    # Pool size of 3 means only 3 sleep commands run at once; others 
    runner = CommandRunner(max_workers=3)

    print(f"[{time.strftime('%H:%M:%S')}] Starting test: Submitting 5

    try:
        for i in range(1, 6):
```

```
    finally:
        print("\nWaiting for all running/queued tasks to complete...")
        runner.shutdown()
        print("All tasks finished. Resources released.")
```

Use code with caution.

## What this test demonstrates:

- **Overlapping Execution**: Tasks are submitted every 2 seconds but take 5 seconds to finish. You will see multiple "SUBMITTING" logs before the first "FINISHED" log appears.
- **Queue Management**: With `max_workers=3`, the 4th and 5th tasks will automatically stay in the internal queue of the ProcessPoolExecutor until one of the first three processes becomes idle.
- **Non-blocking Submission**: The `runner.submit_task` call returns immediately, allowing the loop to continue its 2-second interval sleep regardless of how long the command actually takes.
- **Automatic Callbacks**: The `task_finished_callback` fires independently for each task as soon as it completes, providing the returned output from the subprocess.

Do you need to implement a way to **track the status** of these queued tasks before they finish?

AI responses may include mistakes. Learn more

1 site

To make the test interactive while tasks run in the background, we can use a `while True` loop to capture keyboard input. Since `input()` is a blocking call, the `ProcessPoolExecutor` will continue to process the queue and fire callbacks in its own background threads while the main thread waits for your keystrokes.

Here is the updated interactive test:

python

```python
import time
import subprocess
from concurrent.futures import ProcessPoolExecutor

class CommandRunner:
    def __init__(self, max_workers=None):
        self.executor = ProcessPoolExecutor(max_workers=max_workers)

    def _execute_command(self, cmd_string):
        # Execute the string in a shell; universal_newlines for strin
        result = subprocess.run(
            cmd_string, shell=True, stdout=subprocess.PIPE,
            stderr=subprocess.PIPE, universal_newlines=True
        )
        return {"cmd": cmd_string, "stdout": result.stdout.strip()}

    def submit_task(self, cmd_string, callback=None):
        future = self.executor.submit(self._execute_command, cmd_stri
        if callback:
            future.add_done_callback(callback)
        return future

    def shutdown(self):
        self.executor.shutdown(wait=True)
```

AI Mode All Images Videos News More

```
# Map inputs to specific command strings
tasks = {
    "1": "sleep 5 && echo 'Short Task'",
    "2": "sleep 10 && echo 'Medium Task'",
    "3": "sleep 15 && echo 'Long Task'"
}

print("--- Interactive Task Runner ---")
print("1: 5s task | 2: 10s task | 3: 15s task | q: Quit")

try:
    while True:
        choice = input("Enter 1, 2, 3 (or 'q' to quit): ").strip(

        if choice.lower() == 'q':
            break

        if choice in tasks:
            cmd = tasks[choice]
            print(f"[QUEUED] {cmd}")
            runner.submit_task(cmd, callback=on_finished)
        else:
            print("Invalid input. Use 1, 2, 3 or q.")

except KeyboardInterrupt:
    pass
finally:
    print("\nShutting down... waiting for active tasks to finish.
    runner.shutdown()
    print("Goodbye.")
```

Use code with caution.

- **Asynchronous Notification**: The `on_finished` callback prints the result as soon as the **subprocess** exits, even while you are sitting at the input prompt.

Would you like to add a **unique ID** to each task so you can track which specific submission is finishing?

AI responses may include mistakes. Learn more